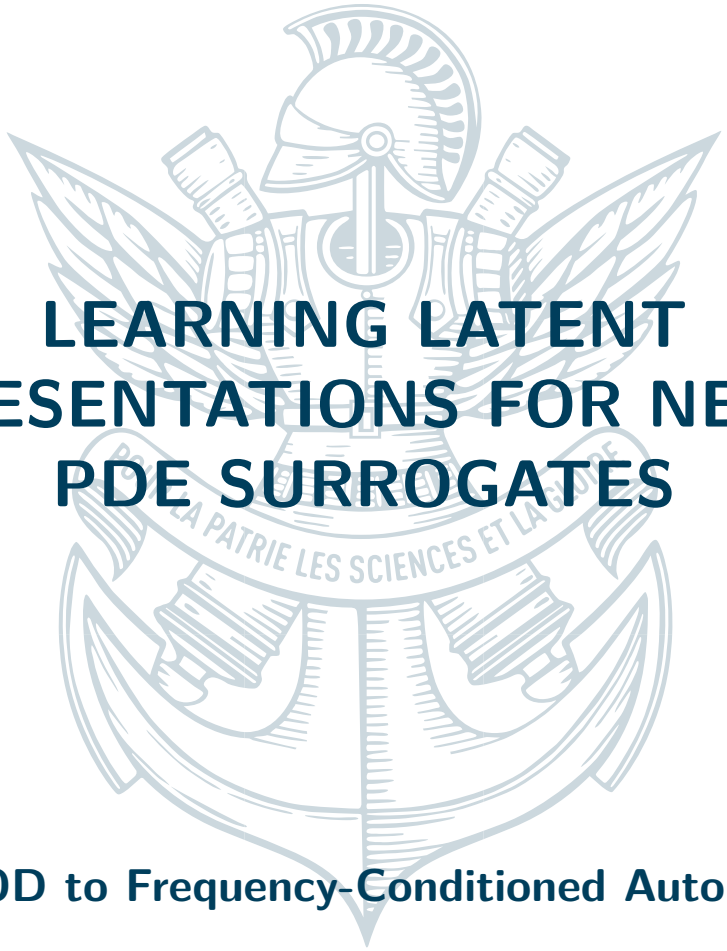


LEARNING LATENT REPRESENTATIONS FOR NEURAL PDE SURROGATES

From POD to Frequency-Conditioned Autoencoders

April 15, 2026

Keyvan Attarian

Contents

1 Introduction

The numerical simulation of physical systems governed by partial differential equations (PDEs) underpins a vast range of scientific and engineering disciplines, from computational fluid dynamics and structural mechanics to atmospheric modelling and chemical process engineering. High-fidelity solvers—whether based on finite elements, finite volumes, or spectral methods—can capture complex multi-scale phenomena with great accuracy, but at a computational cost that renders them impractical for applications requiring thousands or millions of forward evaluations. Real-time decision making, Bayesian inference over large parameter spaces, optimal experimental design, and digital twin frameworks all fall into this category: each query to the solver may take minutes to hours, making exhaustive exploration computationally intractable [1, 7].

This tension between accuracy and computational cost has motivated a rapidly growing body of work on *surrogate modelling* (also referred to as emulation or metamodelling), in which a cheaper approximate model replaces the high-fidelity solver for a given parameter range. Classical surrogate approaches, such as Gaussian processes or polynomial chaos expansions, offer rigorous uncertainty estimates but scale poorly with input dimensionality and struggle to capture the high-dimensional structure of field-valued outputs. Data-driven methods based on deep learning have emerged as a compelling alternative, offering both the flexibility to represent complex non-linear maps and the ability to produce full spatial fields as outputs [4, 7].

A particularly productive paradigm in this context is *physics-informed machine learning*, which seeks to embed physical knowledge—whether through governing equations, symmetry constraints, or conservation laws—directly into the learning architecture or loss function [4, 5]. The present work draws on this philosophy by leveraging the structure of the Laplace transform as a physically motivated temporal basis, decomposing transient fields into independent frequency components that are individually amenable to surrogate modelling.

Application context. The motivating application is the emulation of atmospheric dispersion models for pollutant concentration prediction, including methane (CH4) [2, 3]. Replacing such simulations with fast neural surrogates accelerates real-time monitoring and inverse source identification.

Central contribution. The high dimensionality of field-valued outputs—up to $N_t \times N^2$ degrees of freedom per simulation—makes direct regression intractable. The central approach is to first learn a compact latent representation of the solution fields, either through linear dimensionality reduction (POD, SVD) or through non-linear autoencoders, and then train a surrogate that maps physical parameters $\boldsymbol{\theta}$ into this latent space. This decoupling reduces the surrogate output dimensionality by orders of magnitude while allowing each component to be trained and validated independently.

Outline. Section 2 introduces the framework on a stationary 2-D convection-diffusion problem. Section 3 presents the transient CH4 problem, comparing linear SVD-based latent representations against a non-linear frequency-conditioned autoencoder, followed by the full surrogate pipeline and a residual correction post-processor.

2 Preliminary Study: Stationary Dataset

2.1 Problem Statement and Dataset

The stationary test case is a convection-diffusion problem on the unit square $\Omega = [0,1]^2$:

$$-D\,\Delta U + \boldsymbol{b}\cdot\nabla U = f \quad \text{in } \Omega, \qquad U = 0 \quad \text{on } \partial\Omega, \tag{1}$$

where $D > 0$ is the diffusivity, $\boldsymbol{b} = (b_x, b_y)$ is the convection velocity, and f is a point-source intensity. The parameter vector $\boldsymbol{\theta} = (D, b_x, b_y, f) \in \mathbb{R}^4$ fully specifies a solution instance, and the surrogate task is to learn the forward map $\boldsymbol{\theta} \mapsto U \in \mathbb{R}^{N\times N}$.

The dataset is generated by uniformly sampling $\boldsymbol{\theta}$ over the parameter domain and solving (1) using P1 finite elements with SUPG/GLS stabilisation (FEniCS/DOLFINx). Solutions are interpolated onto a uniform $N \times N$ grid. For machine learning, each field U is centred and rescaled to $[-1, 1]$ using per-pixel statistics computed on the training set; the parameters $\boldsymbol{\theta}$ are standardised using pre-computed mean and standard deviation. Representative examples are shown in Figure 1.

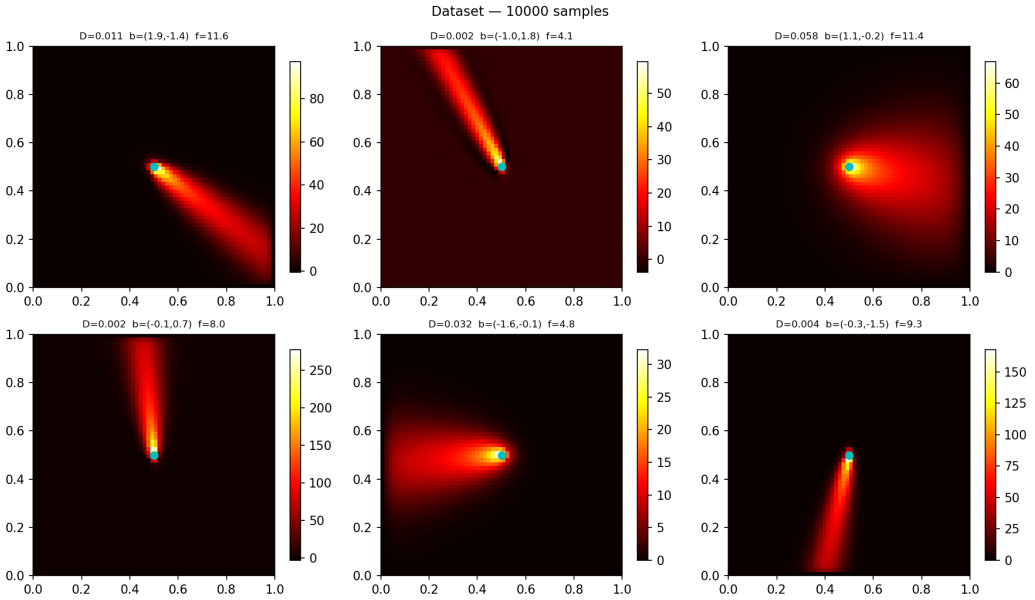


Figure 1: Representative sample of the generated dataset.

2.2 Model

2.2.1 Variational Autoencoder

The first stage trains a variational autoencoder (VAE) to compress solution fields $U \in \mathbb{R}^{N\times N}$ into a low-dimensional latent code $z \in \mathbb{R}^{d_z}$ ($d_z = 32$).

Encoder. The encoder $q_\phi(z|U)$ processes the input field through four strided convolutional blocks (channels $1 \to 16 \to 32 \to 64 \to 128$, kernel 4×4, stride 2, LeakyReLU activations), yielding a total downsampling factor of 16× and a flattened feature vector of size $128 \cdot (N/16)^2$. Two parallel fully-connected heads (Linear → ReLU → Linear) then produce the mean $\mu \in \mathbb{R}^{d_z}$ and log-variance $\log\sigma^2 \in \mathbb{R}^{d_z}$ of the approximate posterior $q_\phi(z|U) = \mathcal{N}(\mu, \sigma^2 I)$. During training the latent code is sampled via the reparameterisation trick $z = \mu + \sigma \odot \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$; at inference the deterministic mean $z = \mu$ is used.

Decoder. The decoder $p_\psi(U|z)$ maps z back to the spatial grid through a linear projection to $128 \cdot b^2$ features (where $b = N/32$), followed by four transposed-convolutional blocks (channels $128 \to 64 \to 32 \to 16 \to 1$, stride 2, BatchNorm + ReLU) that upsample to $(1, N/2, N/2)$. A final dense block (Linear $\to$ ReLU $\to$ Linear) maps the flattened activations to the full N^2 pixel grid and is reshaped to $(1, N, N)$; this fully-connected refinement step allows the network to mix information across all spatial positions, mitigating the checkerboard artefacts that transposed convolutions are prone to introduce.

Training objective. The model is trained by maximising the evidence lower bound (ELBO):

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q(z|U)}\left[\|U - \hat{U}\|^2 + \mathcal{L}_{\text{grad}}(U, \hat{U})\right] + \beta\,\text{KL}\big(q(z|U)\,\|\,\mathcal{N}(0, I)\big), \tag{2}$$

where the reconstruction term penalises pixel-wise squared errors. The gradient consistency term $\mathcal{L}_{\text{grad}}$ applies a spatially weighted penalty on finite-difference approximations of ∂_x and ∂_y, with per-location weights proportional to the ground-truth gradient magnitude; this focuses the penalty on sharp fronts characteristic of convection-dominated regimes. The KL divergence is computed with a *free-bits* threshold $\lambda_{\text{fb}} = 0.1$ per latent dimension: dimensions whose per-dimension KL falls below this threshold are not penalised, preventing posterior collapse. To stabilise training, β is linearly annealed from 0 to its target value over the first 80 epochs.

2.2.2 Indirect Decoder $\boldsymbol{\theta} \to z \to U$

The second stage decouples surrogate training from representation learning. Once the VAE is trained, its decoder p_ψ is frozen and only a lightweight projection network is optimised.

Architecture. The projection network is a two-layer MLP:

$$\boldsymbol{\theta}_{\text{norm}} \xrightarrow{\text{Lin}_{4\to128}} \text{ReLU} \xrightarrow{\text{Lin}_{128\to d_z}} \hat{z},$$

where $\boldsymbol{\theta}_{\text{norm}} \in \mathbb{R}^4$ denotes the standardised parameter vector. The field estimate is then obtained by passing $\hat{z}$ through the frozen VAE decoder: $\hat{U} = p_\psi(\hat{z})$.

Training objective. The projection network is trained end-to-end (VAE decoder frozen) by minimising:

$$\mathcal{L}_{\text{dec}} = \|U - \hat{U}\|^2 + \frac{1}{2}\big(\|\partial_x\hat{U} - \partial_x U\|^2 + \|\partial_y\hat{U} - \partial_y U\|^2\big), \tag{3}$$

combining pixel-level fidelity with an unweighted spatial gradient consistency term. The gradient penalty ensures that the predicted field reproduces the sharp spatial features learned by the frozen decoder, even though the MLP only operates in the low-dimensional latent space. The same AdamW optimiser configuration is used as in Stage 1.

2.3 Results

The VAE is evaluated on 1 000 held-out test samples (10% of the full dataset of 10 000 simulations) that were never seen during training. Reconstruction quality is measured by the relative L^2 error

$$\varepsilon_{\text{rel}} = \frac{\|U - \hat{U}\|_2}{\|U\|_2} \times 100\%.$$

Quantitative performance. Figure 2 shows the distribution of ε_{rel} over the test set. The VAE achieves a **median error of 8.1%** and a mean error of 11.5% (std 9.8%). The distribution is right-skewed: the bulk of samples is reconstructed with errors below 15%, but a long tail extends to high values, corresponding to parameter configurations where the convection is strong (large $|\boldsymbol{b}|$) and produces sharp concentration gradients that are difficult to represent in the 32-dimensional latent code.

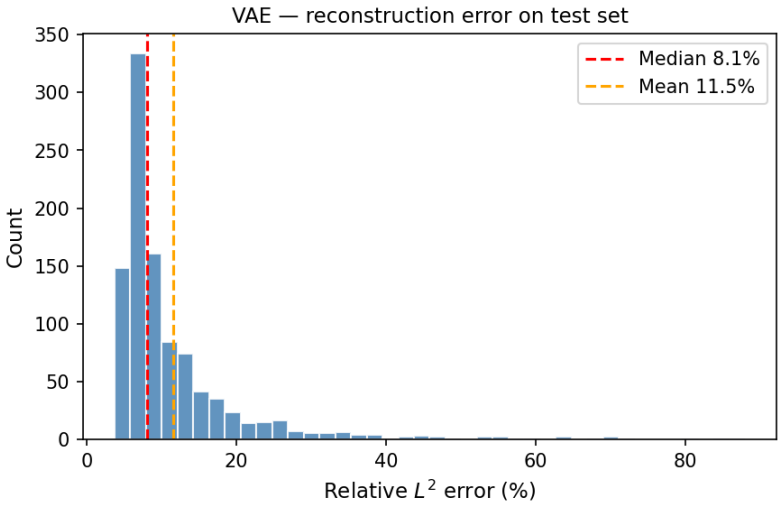


Figure 2: Distribution of relative L^2 reconstruction error on the 1 000-sample test set (median 8.1%, mean 11.5%).

Qualitative reconstruction. Figure 3 displays ground-truth fields alongside VAE reconstructions and pointwise absolute errors on three randomly selected test samples. The autoencoder recovers the global field structure faithfully, including the asymmetric concentration plume driven by the convection velocity. A recurring failure mode is the appearance of faint spurious *rays* in the reconstructed field — traces of convection directions present in other training configurations that bleed into the reconstruction through the shared latent space. This is a manifestation of regression to the mean: the decoder, having seen many plumes oriented in different directions, produces a weighted average of nearby configurations rather than committing to the sharp, direction-specific gradient of the target. The effect is most visible when the convection vector $\boldsymbol{b}$ places the solution near the boundary of a high-density region of the training distribution, where the latent code is pulled towards the centroid of a cluster of neighbouring configurations.

Latent space structure. Figure 4 shows a PCA projection of the 32-dimensional latent codes μ for all test samples, coloured by each physical parameter. The first two principal components capture a substantial fraction of the latent variance. A clear continuous gradient is visible along the first PC when coloured by the diffusivity D and the source intensity f, indicating that the encoder organises the latent space in a physically meaningful manner: samples with similar physical parameters occupy adjacent regions, which is a prerequisite for the surrogate $\boldsymbol{\theta} \to z$ trained in Stage 2 to generalise smoothly.

Indirect decoder evaluation. Four variants of the indirect decoder are evaluated on the same 1 000-sample test set, varying the latent dimension ($d_z \in \{3, 32\}$) and whether the VAE decoder is kept frozen or jointly fine-tuned end-to-end during surrogate training. Table 1 summarises the results; Figure 5 shows the corresponding error distributions alongside the VAE reconstruction floor.

Two findings stand out. First, keeping the decoder frozen yields poor surrogate performance (median 27–33%): the MLP must map $\boldsymbol{\theta}$ to a latent code that lies precisely in the region of $\mathbb{R}^{d_z}$ that the frozen decoder can process accurately, which is a highly constrained and non-trivial regression target. Second, unlocking the decoder for end-to-end fine-tuning dramatically reduces the error: the $d_z = 32$ variant reaches a median of **9.9%**, only 1.8 percentage points above the VAE reconstruction floor of 8.1%. This near-coincidence is the central result: the MLP regression $\boldsymbol{\theta} \mapsto z$ contributes only marginally to the total error. The dominant source of error is the VAE bottleneck itself — the same spurious rays and gradient smoothing already identified in the VAE qualitative analysis above. In other words, the surrogate does not introduce significant new errors; it essentially inherits the imperfections of the latent representation. The $d_z = 3$ variant, while substantially

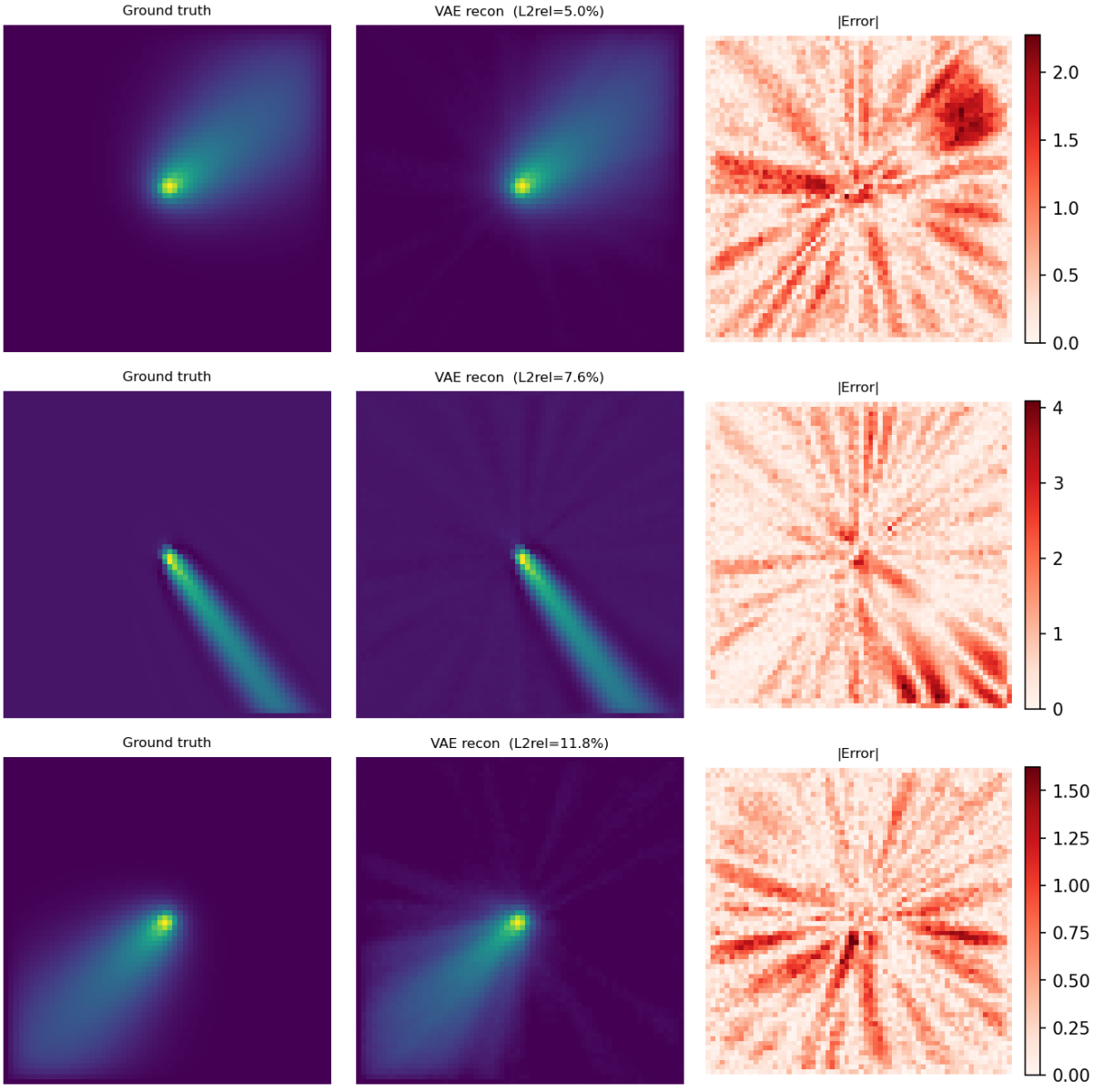


Figure 3: VAE reconstruction on three randomly selected test samples. Left: ground truth. Centre: reconstruction. Right: pointwise absolute error. Residuals concentrate along steep gradient fronts.

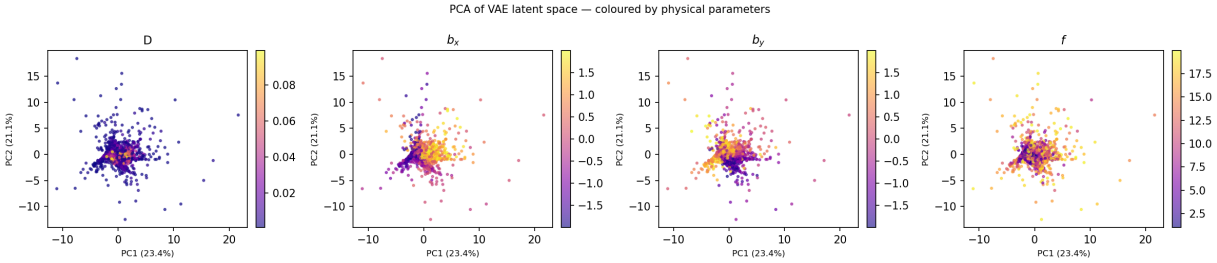


Figure 4: PCA projection of the VAE latent space coloured by the four physical parameters D, b_x, b_y, f. Each point is one test sample. Smooth colour gradients indicate that the encoder maps the parameter space continuously into the latent space.

Model	d_z	Decoder	Median (%)	Mean (%)
VAE (reconstruction floor)	32	—	8.1	11.5
Indirect decoder	32	frozen	27.7	37.0
Indirect decoder	32	end-to-end	**9.9**	**13.2**
Indirect decoder	3	frozen	32.8	47.2
Indirect decoder	3	end-to-end	14.0	18.8

Table 1: Relative L^2 error on the 1 000-sample test set for all surrogate variants. The VAE reconstruction floor is included as reference.

better than its frozen counterpart, remains at 14.0% because the three-dimensional bottleneck discards too much spatial information for any surrogate to recover, regardless of how accurate the $\boldsymbol{\theta} \mapsto z$ regression is.

Figure 6 confirms this qualitatively: the spatial structure of the surrogate errors is indistinguishable from that of the VAE errors in Figure 3 — the same spurious directional rays, the same gradient smoothing at sharp fronts. The surrogate does not introduce new artefacts; it faithfully propagates the limitations of the latent representation. Improving the surrogate accuracy therefore requires improving the autoencoder, not the MLP.

2.4 Discussion

The two-stage pipeline performs well on the stationary problem: the end-to-end surrogate reaches 9.9% median error, close to the 8.1% VAE floor. Further architectural improvements are possible but not the primary focus of this work. The key takeaway is that a convolutional autoencoder can reliably organise a compact, physically structured latent space from PDE solution fields — giving us confidence that the same strategy, extended to the spectral structure of transient data, will provide a solid foundation for the more demanding surrogate developed

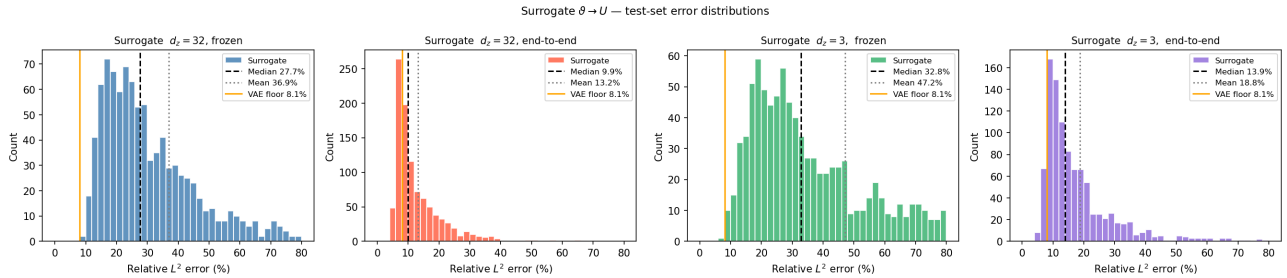


Figure 5: Test-set error distributions for the four surrogate variants. The orange vertical line marks the VAE reconstruction floor (median 8.1%). End-to-end fine-tuning brings the $d_z = 32$ surrogate close to this floor.

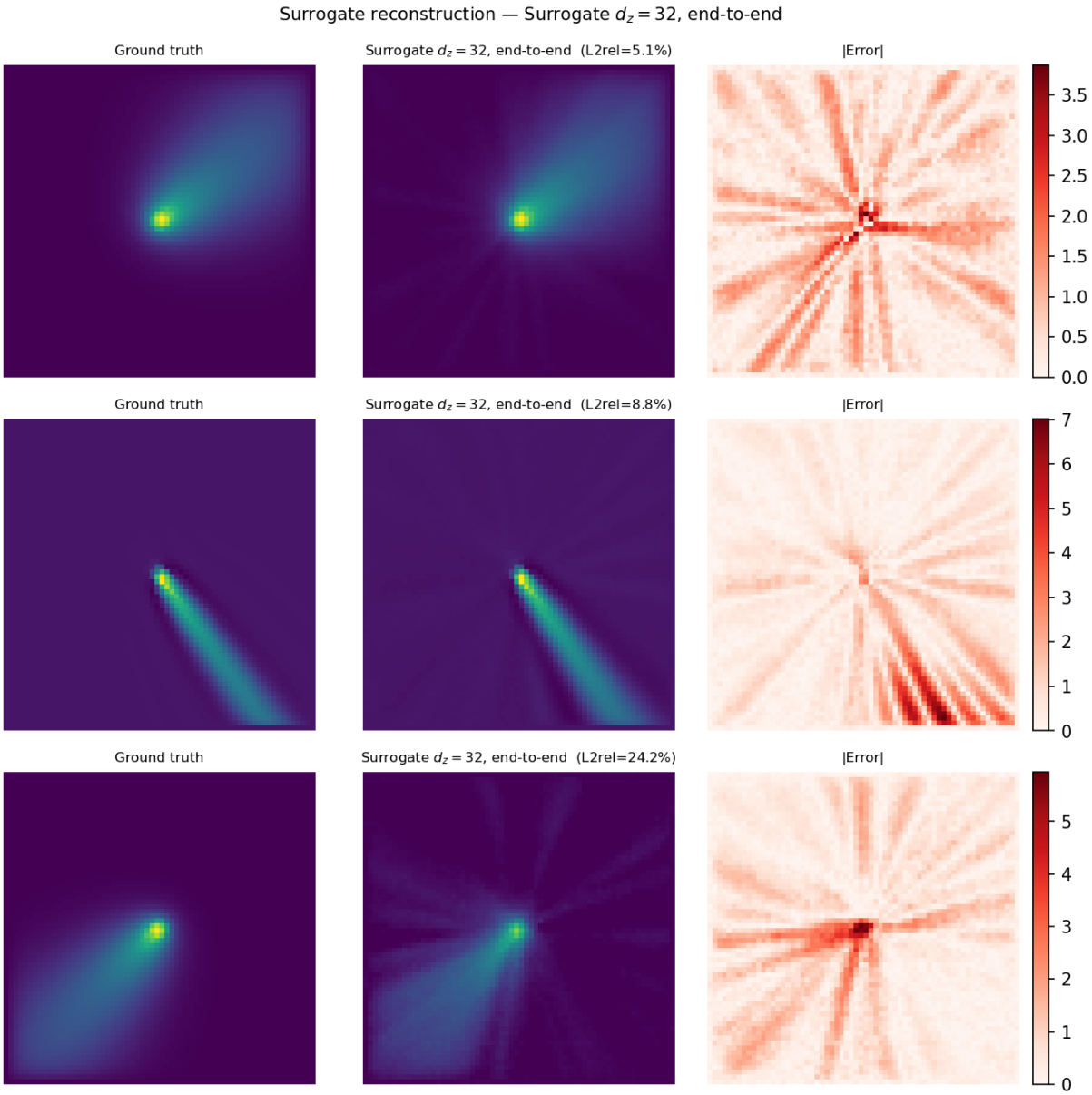


Figure 6: Surrogate reconstruction ($d_z = 32$, end-to-end) on three randomly selected test samples. Residuals concentrate along steep gradient fronts, consistent with the VAE reconstruction errors.

in the following sections.

3 Transient CH4 Concentration Fields

3.1 Dataset and Temporal Representation

3.1.1 Dataset

The transient experiments use a dataset of 8 100 simulations of atmospheric methane (CH_4) concentration fields. Each simulation yields a time series of 150 snapshots on a 200×200 spatial grid, forming a tensor of shape $(8100, 150, 200, 200)$. The physical parameter vector is $\boldsymbol{\theta} = (k, A, C)$, representing the reaction rate, source amplitude, and initial concentration respectively. To accommodate the ~ 17 GB total dataset size, all data are loaded and processed using memory-mapped arrays, avoiding materialisation of the full tensor in RAM at any point.

Figure 7 shows a representative example of the transient field evolution over time. The dataset captures an exothermic combustion reaction: methane released from a point source undergoes rapid oxidation, releasing energy. This chemical reaction drives sharp concentration gradients and fast transient dynamics early in the evolution. A critical challenge for the surrogate is the presence of an *explosion-like* phenomenon: the initial reaction creates localised high-intensity regions with steep spatial gradients that evolve rapidly. Capturing this violent early-time behaviour requires both fine spatial resolution (high-frequency modes to represent the sharp concentration fronts) and accurate temporal dynamics (to predict the reaction-driven evolution). Surrogates that fail to resolve the combustion transient will accumulate error early in the simulation, which then propagates through subsequent diffusion and dilution phases, compounding the overall prediction error.

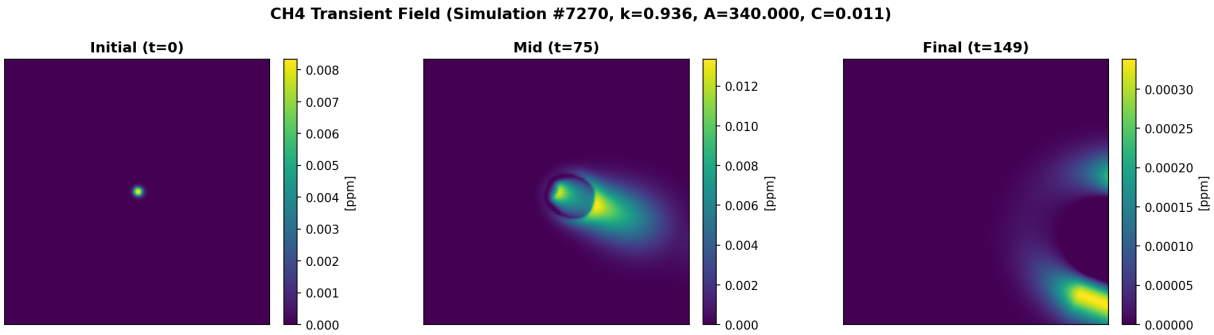


Figure 7: Transient CH_4 concentration field at three snapshots: initial (t=0), mid-time (t=75), and final (t=149). The field evolves from a concentrated plume to a dispersed distribution, illustrating the combined effect of atmospheric diffusion and chemical reactions.

3.1.2 Laplace Transform

Rather than operating in physical time, the surrogate works in the Laplace frequency domain. The Laplace transform of a time-dependent field is approximated via a discretised Bromwich contour integral, computed efficiently using the discrete Fourier transform with exponential damping:

$$\hat{U}(s_k) = \int_0^T U(t)\, e^{-s_k t}\, \mathrm{d}t \approx \mathrm{DFT}\big[U(t) \cdot w_t \cdot e^{-\gamma t}\big], \qquad s_k = \gamma + i\omega_k, \tag{4}$$

where w_t are trapezoidal integration weights and $\gamma \geq 0$ is a damping parameter that improves numerical stability. The spatial field U is transformed at each frequency s_k into a complex-valued field $\hat{U}(s_k) \in \mathbb{C}^{N \times N}$. For real-valued physical fields, Hermitian symmetry provides $\hat{U}(s_{N_t - k}) = \overline{\hat{U}(s_k)}$, so only $N_{\text{half}} = N_t/2 + 1$ frequencies need be stored. In practice, each frequency s_k is represented as the pair of real fields $(\Re(\hat{U}_k), \Im(\hat{U}_k)) \in \mathbb{R}^{2 \times N \times N}$.

The surrogate models only the first $k_{\max}$ frequencies (specifically, $k_{\max} = 20$), which typically capture the essential dynamics while reducing computational cost. Higher frequencies ($k > k_{\max}$) are set to zero in the

normalised representation, corresponding to the mean field after denormalisation. Inversion to physical time is performed by discrete Fourier inversion with exponential compensation on the full reconstructed spectrum.

3.2 Latent Space Representation

3.2.1 Tucker POD Surrogate

Algorithm. The Tucker POD method, introduced by Ammar *et al.* [1], computes a greedy rank-$(1,1,1)$ Tucker decomposition of the full training tensor by sequentially extracting separable modes. Given the three-dimensional tensor $\boldsymbol{H} \in \mathbb{R}^{n_r \times n_s \times N_t}$, the decomposition seeks an approximation of the form

$$H_{r,s,t} \approx \sum_{j=1}^{n_f} \alpha_j \, F_{r,j} \, G_{s,j} \, P_{t,j}, \tag{5}$$

where $\boldsymbol{F} \in \mathbb{R}^{n_r \times n_f}$ collects spatial modes, $\boldsymbol{P} \in \mathbb{R}^{N_t \times n_f}$ collects temporal modes, $\boldsymbol{G} \in \mathbb{R}^{n_s \times n_f}$ collects simulation-dependent coefficients, and $\boldsymbol{\alpha} \in \mathbb{R}^{n_f}$ are the corresponding singular values. Each mode j is extracted from the current residual tensor $\boldsymbol{R}^{(j)} = \boldsymbol{H} - \sum_{i<j} \alpha_i \boldsymbol{f}_i \otimes \boldsymbol{g}_i \otimes \boldsymbol{p}_i$ by solving a rank-1 approximation problem via alternating least squares (ALS). Starting from random vectors R, S, T, the following updates are iterated until convergence:

$$R \leftarrow \frac{\boldsymbol{R}^{(j)}_{(1)} \, (T \otimes S)}{\|S\|^2 \, \|T\|^2}, \qquad S \leftarrow \frac{\boldsymbol{R}^{(j)}_{(2)} \, (R \otimes T)}{\|R\|^2 \, \|T\|^2}, \qquad T \leftarrow \frac{\boldsymbol{R}^{(j)}_{(3)} \, (S \otimes R)}{\|R\|^2 \, \|S\|^2}, \tag{6}$$

where $\boldsymbol{R}^{(j)}_{(k)}$ denotes the mode-k unfolding of the residual tensor. Once convergence is reached, the amplitude is $\alpha_j = \|R\| \, \|S\| \, \|T\|$, the normalised vectors $\boldsymbol{f}_j = R/\|R\|$, $\boldsymbol{g}_j = S/\|S\|$, $\boldsymbol{p}_j = T/\|T\|$ are stored, and the deflation $\boldsymbol{R}^{(j+1)} \leftarrow \boldsymbol{R}^{(j)} - \alpha_j \boldsymbol{f}_j \otimes \boldsymbol{g}_j \otimes \boldsymbol{p}_j$ is applied. The algorithm terminates when the relative residual $\|\boldsymbol{R}^{(j)}\|_F / \|\boldsymbol{H}\|_F$ falls below a prescribed tolerance, or after n_f modes have been extracted.

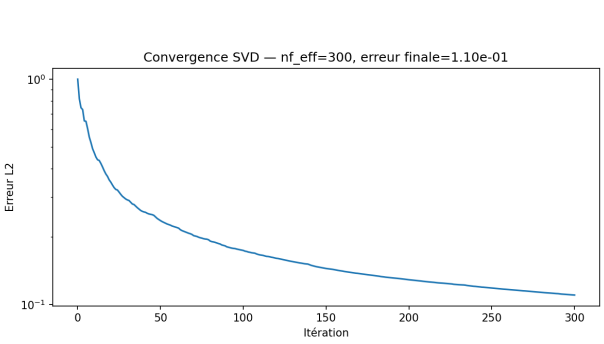
Application to the CH4 dataset. The decomposition is applied to the full training set of $n_s = 8\,100$ simulations. Because the algorithm requires the entire tensor to reside in memory simultaneously, the spatial grid is subsampled with stride $\delta = 5$, reducing each 200×200 frame to a 40×40 grid ($n_r = 1\,600$ nodes). Up to $n_f = 300$ modes are extracted; the convergence of the relative residual as a function of mode index is shown in Figure 8(a).

Surrogate training. The spatial factors $\boldsymbol{F}$, temporal factors $\boldsymbol{P}$, and singular values $\boldsymbol{\alpha}$ are fixed after the Tucker decomposition and stored in the checkpoint. The simulation-dependent coefficient vector $\boldsymbol{g} \in \mathbb{R}^{n_f}$ (the s-th row of $\boldsymbol{G}$) encodes the identity of each simulation in a compact latent space. A four-layer MLP is trained to predict these coefficients from the physical parameters: $\boldsymbol{\theta}_{\text{norm}} \mapsto \hat{\boldsymbol{g}} \in \mathbb{R}^{n_f}$. At inference time, the full spatiotemporal field on the subsampled grid is reconstructed as

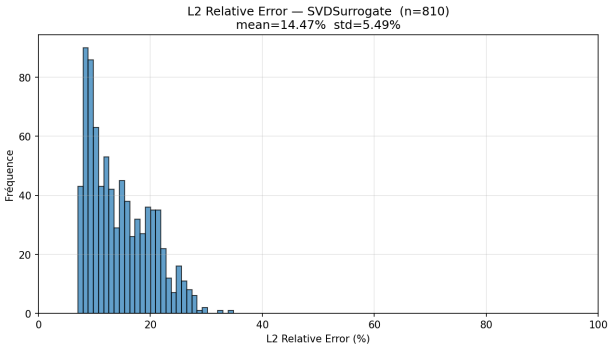
$$\hat{H}_{r,t} = \sum_{j=1}^{n_f} \alpha_j \, F_{r,j} \, \hat{g}_j \, P_{t,j}.$$

Limitations. The test-set error reported in Figure 8(b) conflates two independent sources of error. First, the Tucker decomposition itself introduces a *representation error*: the best rank-n_f approximation of $\boldsymbol{H}$ is not exact, and the relative residual after convergence is approximately 10%. This is a floor that no surrogate, however accurate, can overcome. Second, the MLP must learn a non-linear map $\boldsymbol{\theta} \mapsto \boldsymbol{g}$; any error in predicting the coefficient vector propagates through the reconstruction formula and adds to the representation error. The observed test-set median error of around 30–40% thus reflects the cumulation of these two contributions.

A more fundamental constraint stems from the batch nature of the algorithm: equations (6) require the *entire* training tensor $\boldsymbol{H}$ to reside in GPU memory simultaneously. With $n_r = 1\,600$, $n_s = 8\,100$, and $N_t = 150$ in single precision, this amounts to approximately 7.8 GB. Increasing the spatial resolution directly scales this



(a) Reconstruction error vs. number of retained modes n_f.



(b) Relative L^2 error on the test set (surrogate).

Figure 8: Tucker POD surrogate: convergence of the Tucker decomposition (a) and final prediction error on the test set (b).

requirement: restoring the full 200×200 grid would require $25\times$ more memory ($\sim$ 193 GB), which is infeasible. Consequently, the Tucker POD approach is confined to the downsampled 40×40 representation, which discards spatial details at scales finer than five grid cells. This resolution bottleneck, combined with the non-negligible representation error, motivates the exploration of surrogate methods that operate in a transformed space where the dimensionality is more manageable.

3.2.2 SVD Surrogate in the Laplace Domain

An alternative linear approach applies truncated SVD independently to each Laplace frequency. For frequency k, the real and imaginary parts of $\hat{U}_k$ across all training samples are arranged into a matrix, and the leading k_{svd} right singular vectors $\boldsymbol{V}^{(k,r)}$ and $\boldsymbol{V}^{(k,i)}$ are extracted:

$$\Re(\hat{U}_k) \approx \boldsymbol{c}^{(k,r)} \big(\boldsymbol{V}^{(k,r)}\big)^\top, \qquad \Im(\hat{U}_k) \approx \boldsymbol{c}^{(k,i)} \big(\boldsymbol{V}^{(k,i)}\big)^\top.$$

The spatial basis $\boldsymbol{V}$ is fixed during training. For each frequency k, a dedicated three-layer MLP learns to predict the normalised coefficients $\boldsymbol{c}^{(k,r)}$ and $\boldsymbol{c}^{(k,i)}$ from the physical parameters. Field reconstruction follows by projecting the predicted coefficients onto the fixed bases; the full time series is then recovered by inverting the Laplace transform as in (4).

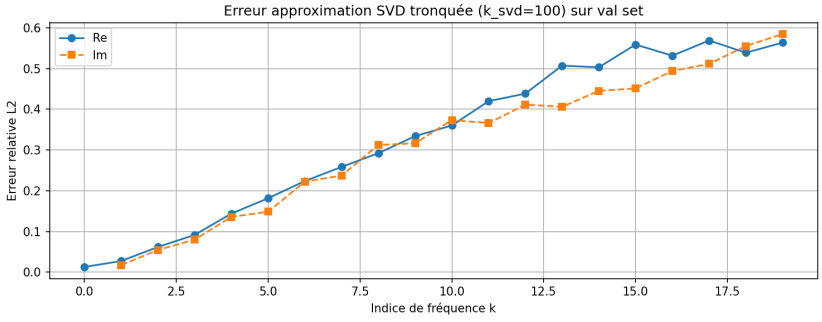


Figure 9: SVD reconstruction error vs. k_{svd} for the per-frequency SVD approach.

While the per-frequency SVD factorisation operates on the full 200×200 spatial resolution (circumventing the memory bottleneck of Tucker POD), it faces its own irreducible limitation: the linear basis truncation error is already substantial at the lowest frequencies (5–10%) and grows monotonically with frequency. Consequently, the MLP surrogates are trained on noisy targets that are only approximate representations of the true Laplace

fields. The combination of fixed linear bases with inherent reconstruction loss and complex non-linear prediction tasks yields poor generalisation on the test set, motivating the transition to a learned, non-linear autoencoder.

3.2.3 Autoencoder-Based Latent Space

Both SVD approaches above constrain the surrogate to lie in a fixed linear subspace extracted from the training data. When Laplace fields exhibit non-linear variation across the parameter space, this linearity becomes restrictive. To overcome this limitation, a learned autoencoder is trained to extract a non-linear latent representation that is shared across all frequencies. The encoder and decoder are conditioned on the frequency index k, allowing a single pair of networks to process fields at any frequency while adapting their behaviour through frequency-specific control signals.

Sinusoidal frequency encoding. The frequency index k is first normalised to the range $[0, 1]$ by dividing by N_{half}, giving a frequency ratio $f = k/N_{\text{half}}$. This scalar is then embedded into a higher-dimensional vector $\boldsymbol{e}(f) \in \mathbb{R}^{64}$ via sinusoidal basis functions inspired by positional encoding in transformers. The embedding is computed as follows: for each octave $i = 0, 1, \ldots, L-1$, sinusoidal components $\sin(2^i \pi f)$ and $\cos(2^i \pi f)$ are computed, yielding a $2L$-dimensional vector. This vector is passed through a small MLP to produce the final frequency embedding:

$$\boldsymbol{e}(f) = \text{MLP}\Big(\big[\sin(2^i \pi f),\ \cos(2^i \pi f)\big]_{i=0}^{L-1}\Big), \qquad L = 8. \tag{7}$$

This multi-scale representation allows the network to capture frequency information at multiple resolutions simultaneously, enabling fine discrimination among different frequency bands.

FiLM conditioning. To allow the encoder and decoder weights to be shared across all frequencies while adapting their behaviour to each specific frequency, the Feature-wise Linear Modulation (FiLM) mechanism is employed. At each convolutional layer ℓ of the encoder and decoder, the frequency embedding $\boldsymbol{e}(f)$ is passed through two learned linear projections to produce a channel-wise gain vector $\gamma_\ell(\boldsymbol{e}(f)) \in \mathbb{R}^{C_\ell}$ and a channel-wise bias vector $\beta_\ell(\boldsymbol{e}(f)) \in \mathbb{R}^{C_\ell}$, where C_ℓ denotes the number of feature channels at layer ℓ. These gains and biases are used to modulate the intermediate activations $\boldsymbol{x}_\ell$ element-wise:

$$\boldsymbol{x}_\ell \ \leftarrow\ \boldsymbol{x}_\ell \odot \big(1 + \gamma_\ell(\boldsymbol{e}(f))\big) \ +\ \beta_\ell(\boldsymbol{e}(f)). \tag{8}$$

For each channel c, the gain $\gamma_\ell^{(c)}$ applies a multiplicative correction and the bias $\beta_\ell^{(c)}$ applies an additive correction, both functions of the frequency alone. This design allows all frequencies to share the same convolutional weights while the FiLM parameters (derived from the frequency embedding) provide frequency-specific adjustment at each layer.

Architecture and training. The autoencoder encoder takes a two-channel input $(\Re(\hat{U}_k), \Im(\hat{U}_k)) \in \mathbb{R}^{2\times N\times N}$ representing the real and imaginary parts of a Laplace field at frequency k, and progressively downsamples it through strided convolutions to produce a latent code $z \in \mathbb{R}^{d_z}$. Each convolutional block is followed by FiLM modulation using the frequency embedding $\boldsymbol{e}(f)$. The decoder reverses this process, starting from z and upsampling through transposed convolutions, also FiLM-conditioned at each layer. The decoder outputs two separate branches, one predicting $\Re(\hat{U}_k)$ and the other $\Im(\hat{U}_k)$.

Training operates on the Cartesian product of all (simulation, frequency) pairs in the dataset, with the objective:

$$\mathcal{L}_{\text{AE}} = \|\hat{U} - U\|^2 + \beta\,\|\hat{U}\|^2, \tag{9}$$

where the first term is the reconstruction MSE and the second is an L^2 regularisation penalty on the output. To ensure diverse frequency coverage within each mini-batch, (simulation, frequency) pairs are reshuffled at the start of each epoch.

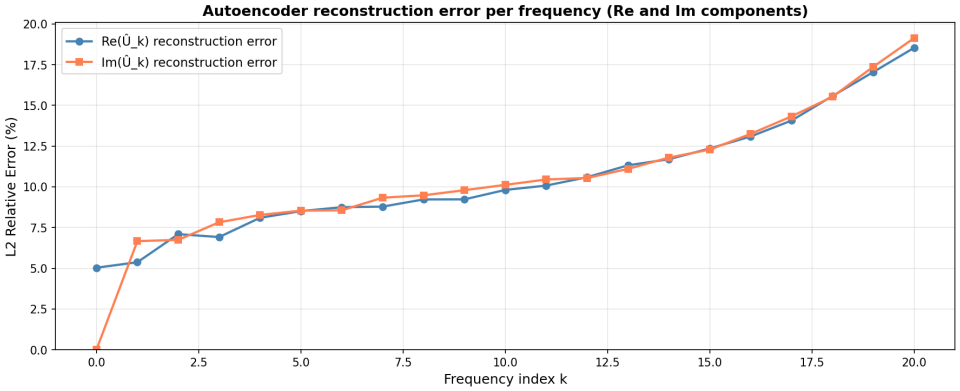


Figure 10: Autoencoder L2 relative reconstruction error per frequency, evaluated on 64 test samples. The AE exhibits increasing error at higher frequencies, reflecting the difficulty of capturing high-frequency oscillations in the latent code.

Autoencoder performance evaluation. The trained autoencoder achieves varying reconstruction accuracy across frequencies. Analysis over 64 test samples reveals the following error profile (Figure 10):

The reconstruction error exhibits a systematic trend across the frequency spectrum:

- **Low frequencies** ($k < 7$): mean error 6.80% (Re) / 6.46% (Im). These frequencies contain the dominant energy and are most accurately captured by the latent representation.
- **Mid frequencies** ($7 \leq k < 14$): mean error 9.42% (Re) / 9.79% (Im). A gradual degradation as frequency increases, requiring larger latent codes to represent finer spatial details.
- **High frequencies** ($k \geq 14$): mean error 13.93% (Re) / 14.04% (Im). Highest reconstruction cost due to oscillatory fine structure. The peak error at $k = 20$ (17.32% Re, 17.64% Im) reflects the challenge of embedding high-frequency ripples in the fixed-size latent code.

Overall, the AE achieves a mean reconstruction error of 10.05% (Re) / 10.09% (Im) across all trained frequencies (median: 9.54% / 9.55%, std: 3.26% / 3.71%). This modest error baseline sets the stage for the parameter-to-latent surrogates: the surrogate will inherit any AE errors at each frequency, and subsequent fine-tuning and post-processing must overcome these irreducible limitations.

To assess the full pipeline quality, the AE is evaluated end-to-end in the temporal domain. For each test sample, frequencies $k \leq k_{\max}$ are reconstructed via the AE; frequencies $k > k_{\max}$ are replaced by their per-pixel training mean $\bar{U}^{(k)}$ (the same value that the surrogate implicitly outputs at those frequencies, since a normalised prediction of zero maps to the training mean after denormalisation). Both the predicted and reference spectra are then inverted to the time domain via (4). Over 64 test samples, this yields a mean temporal L^2 relative error of **9.94%** (median: 9.17%, std: 2.73%, range: [6.68%, 18.61%]), shown in Figure 11. This figure constitutes the irreducible AE floor: no surrogate trained on this latent space can achieve better temporal reconstruction accuracy without also improving the autoencoder.

Choice of $k_{\max}$. The monotone increase in AE error from 6.80% at low frequencies to 13.93% at high frequencies provides an empirical justification for limiting surrogate training to $k_{\max} = 20$ rather than all $N_{\text{half}} = 76$ frequencies. Beyond $k = 20$, the autoencoder reconstruction quality degrades sharply; at these frequencies, any surrogate trained to predict the latent code will inherit substantial baseline errors and face a difficult learning problem. Rather than chasing diminishing returns on high-frequency prediction—where the AE is already making substantial errors—the strategy focuses computational effort on the lower frequencies where the representation is more reliable and the surrogate has clearer signal to learn from. The unmodelled high frequencies are replaced by the per-pixel training mean upon inversion, which, while lossy, avoids training expensive surrogates on irreducibly noisy targets.

Detailed reconstructions at selected frequencies (k = 0, 10, 20) illustrate the spatial structure of AE errors across the frequency range. Low-frequency fields (k=0) exhibit smooth errors localised to regions of strong

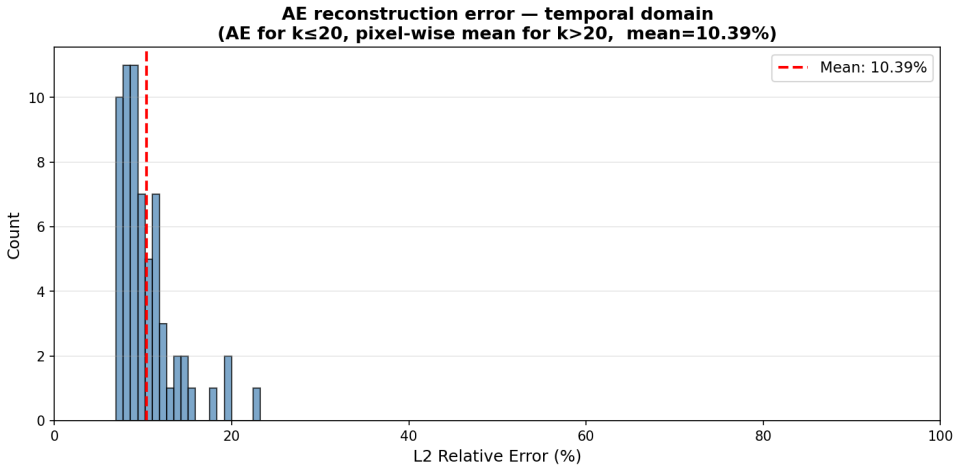
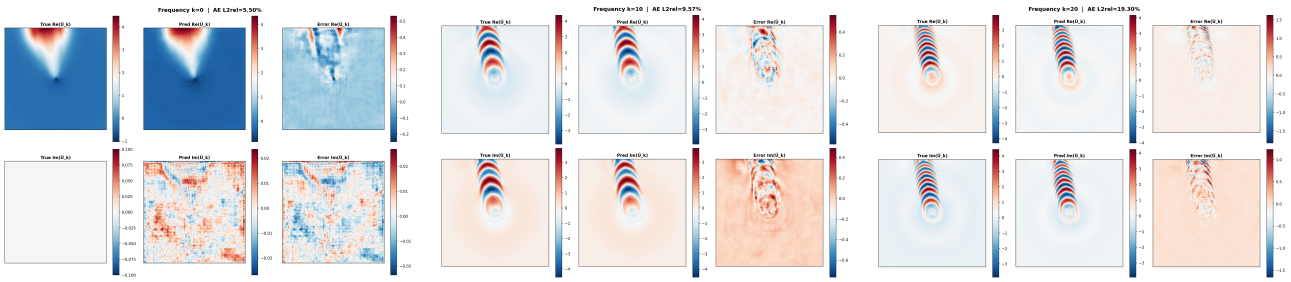


Figure 11: L^2 relative error in the temporal domain: AE predictions for $k \leq k_{\max} = 20$, per-pixel training mean for $k > k_{\max}$, after Laplace inversion. The reference is the full-spectrum Laplace inversion on the same 128×128 grid. Mean error: 9.94%.

gradients, while high-frequency fields (k=20) show more distributed, oscillatory error patterns reflecting the difficulty of capturing fine spatial detail in the fixed-size latent code (Figure 12).



(a) k=0 (DC) (b) k=10 (mid) (c) k=20 (high)

Figure 12: Spatial structure of AE reconstruction errors at selected Laplace frequencies. Each panel shows the true fields (top row) and error fields (bottom row) for both real and imaginary components. Low frequencies exhibit smooth, localised errors; high frequencies show more diffuse, oscillatory patterns.

Limitations of the frequency-conditioned autoencoder. Despite the encouraging reconstruction numbers, this AE design carries several structural limitations that are worth acknowledging. First, the frequency conditioning via FiLM means that there is no single shared latent space: the geometry of z depends on the frequency ratio f_k, so the latent code for frequency $k = 0$ lives in a different effective space than the code for $k = 20$. This prevents any cross-frequency regularisation or interpolation in latent space, and likely makes the manifold less regular than a standard unconditional autoencoder would produce. The per-frequency surrogates therefore face a harder generalisation problem: rather than interpolating on a well-structured shared manifold, each MLP must navigate its own frequency-specific latent geometry. Second, the 9.94% mean temporal reconstruction error is a non-trivial baseline: it represents the minimum error achievable by any surrogate trained on top of this autoencoder, regardless of how well the parameter-to-latent mapping is learned. This irreducible floor is less damaging than it may appear, however, because the architecture is designed with end-to-end fine-tuning in mind. Once the per-frequency surrogates are trained with the decoder frozen, the full model—MLPs and shared decoder jointly—can be fine-tuned end-to-end, allowing the decoder to adapt and reduce the reconstruc-

tion artefacts that the fixed AE introduced. This fine-tuning stage, described in Section 3.3, is precisely what mitigates the AE floor in practice.

3.3 Surrogate and Fine-Tuning

3.3.1 Per-Frequency Surrogate $\boldsymbol{\theta} \to z$

Once the frequency-conditioned autoencoder has been trained, its decoder is frozen, and independent surrogates are trained for each frequency. The surrogate operates only on the first $k_{\max}$ frequencies (with $k_{\max} = 20$); higher frequencies are not explicitly predicted. For frequencies $k = 0, 1, \ldots, k_{\max}$, a four-layer MLP with layer normalisation learns the mapping from normalised physical parameters to the latent code: $\boldsymbol{\theta}_{\text{norm}} \mapsto z_k \in \mathbb{R}^{d_z}$. The shared decoder is kept frozen to preserve the learned latent structure.

At inference time, each predicted code z_k is decoded using the frozen decoder, which is itself conditioned on the frequency ratio f_k via FiLM, yielding predictions for frequencies $k \leq k_{\max}$. Frequencies $k > k_{\max}$ are left at zero in the normalised space, which corresponds to the mean value after denormalisation. The complete Laplace spectrum is then obtained by Hermitian symmetry (the upper frequencies are conjugates of the lower ones), and the time-domain field is recovered by Laplace inversion as in (4). The baseline surrogate (with frozen AE decoder) achieves the error distribution shown in Figure 13.

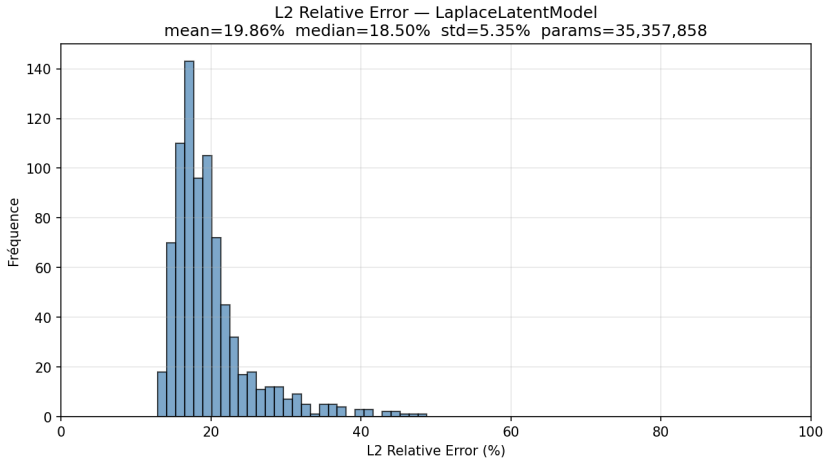


Figure 13: Relative L^2 error distribution for the baseline latent surrogate (mean: 19.86%, median: 18.50%).

3.3.2 End-to-End Fine-Tuning

To further improve accuracy, the frozen decoder is unfrozen and the entire model—both the per-frequency MLPs and the shared decoder—is fine-tuned jointly end-to-end on the training set. Differential learning rates are employed to balance the adaptation: $\eta_{\text{surr}} = 5 \times 10^{-5}$ for the per-frequency parameter-to-latent MLPs and $\eta_{\text{dec}} = 10^{-5}$ for the decoder. The training batches are organised in a frequency-first fashion, processing all simulations for a single frequency in one mini-batch. This structure allows for efficient batching: a single forward call through the FiLM-conditioned decoder handles all batch samples simultaneously, improving computational efficiency compared to a naive per-simulation approach. The results of end-to-end fine-tuning are displayed in Figure 14.

Limitations: Laplace truncation and Gibbs effects. Despite the substantial gains from fine-tuning, a fundamental limitation arises from the truncation of the Laplace spectrum. The surrogate predicts only the first $k_{\max} = 20$ frequencies out of the available $N_{\text{half}} = 51$ frequencies (for $N_t = 150$ snapshots). Higher frequencies are set to zero during inversion, effectively replacing the truncated spectrum with the mean field. This spectral truncation introduces *Gibbs phenomenon*—a well-known oscillatory artefact that appears near sharp transitions

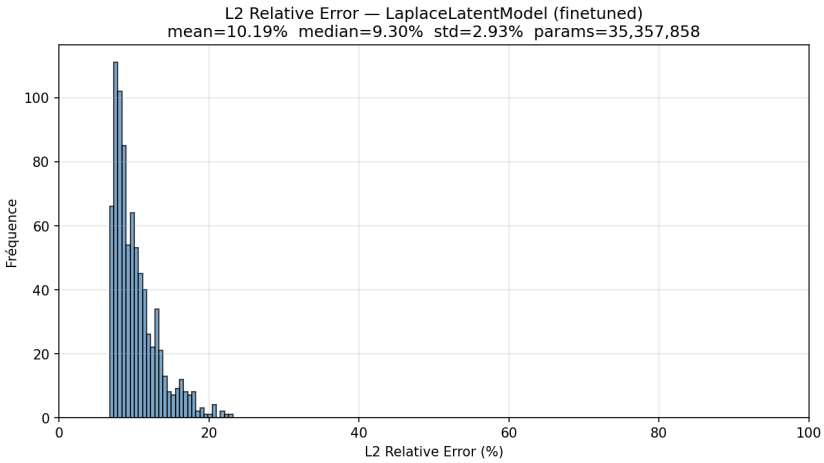


Figure 14: Relative L^2 error distribution after end-to-end fine-tuning (mean: 10.19%, median: 9.30%). Fine-tuning reduces error by approximately 48% compared to the frozen baseline.

in fields reconstructed from truncated orthogonal expansions. The high-frequency details necessary to capture sharp concentration gradients or rapid transients are lost, creating residual ripples that the surrogate cannot overcome. This truncation error manifests as a prediction floor around 6–8% median error, below which further improvements require either: (i) training on more frequencies, (ii) using a larger $k_{\max}$, or (iii) applying non-spectral surrogate approaches. The residual corrector, described next, partially alleviates this limitation by learning to suppress Gibbs oscillations frame by frame.

3.4 Post-Processing and Results

3.4.1 Residual Correction

A UNet residual corrector is applied post hoc, frame by frame, to the surrogate output. A UNet is an encoder–decoder convolutional network in which the encoder progressively downsamples the spatial resolution through successive convolution and pooling operations, while the decoder restores the original resolution through transposed convolutions. At each resolution level, the decoder receives, via a skip connection, the feature maps produced by the encoder at the corresponding level; these are concatenated channel-wise before the next convolution. This architecture allows the network to combine coarse, context-rich features from the bottleneck with fine spatial detail preserved by the skip connections. A standard autoencoder forces all information through the bottleneck, discarding spatial detail that cannot be encoded in the latent code; the skip connections of the UNet bypass this bottleneck, providing the decoder with direct access to high-resolution encoder features and enabling precise spatial localisation in the output.

Given the surrogate prediction $U_{\text{pred}}(t) \in \mathbb{R}^{N\times N}$, the residual correction operates as follows. The predicted field is first normalised per-frame using its own spatial statistics (mean μ and standard deviation σ), then passed through a UNet that predicts a residual correction in the normalised space. The residual is rescaled back to physical space and added to the original prediction:

$$U_{\text{corr}} = U_{\text{pred}} + \sigma(U_{\text{pred}}) \cdot r\left(\frac{U_{\text{pred}} - \mu(U_{\text{pred}})}{\sigma(U_{\text{pred}})}\right), \tag{10}$$

Here $r(\cdot)$ is a three-level UNet with skip connections, employing $c = 16$ base channels and approximately 481k parameters. A crucial design choice is that the final output layer of the UNet is zero-initialised, ensuring that $r \equiv 0$ at the start of training. This initialisation strategy causes the corrected field to equal the surrogate prediction initially, allowing the network to learn only the correction needed rather than the full field.

3.4.2 Training the Corrector

A key practical consideration in training the residual corrector is computational efficiency. Rather than calling the frozen surrogate model repeatedly during training, all $(U_{\text{pred}}, U_{\text{true}})$ pairs are pre-computed and stored on disk as memory-mapped arrays before the corrector training begins. This pre-computation step ensures that no forward pass through the surrogate is needed during the training loop, significantly accelerating convergence.

The corrector loss combines reconstruction accuracy with spatial smoothness. Denoting $r_{\text{pred}} = U_{\text{corr}} - U_{\text{pred}}$ as the predicted residual and $r_{\text{true}} = U_{\text{true}} - U_{\text{pred}}$ as the target residual, the loss function is:

$$\mathcal{L}_{\text{corr}} = \left\| \frac{r_{\text{pred}} - r_{\text{true}}}{\sigma_r} \right\|^2 + \lambda_\nabla \left\| \frac{\nabla r_{\text{pred}} - \nabla r_{\text{true}}}{\sigma_r} \right\|^2, \tag{11}$$

where $\sigma_r = \sigma(r_{\text{true}})$ is the standard deviation of the target residual field, and $\lambda_\nabla = 10$ is the gradient weighting parameter. Both the residual MSE and the spatial gradient MSE are normalised by σ_r, which accounts for the magnitude of the true correction and prevents the loss from being dominated by frames requiring large corrections.

3.4.3 Results and Comparison

The residual correction stage applies the UNet post-processor to the finetuned surrogate output, frame by frame. The error distribution is shown in Figure 15.

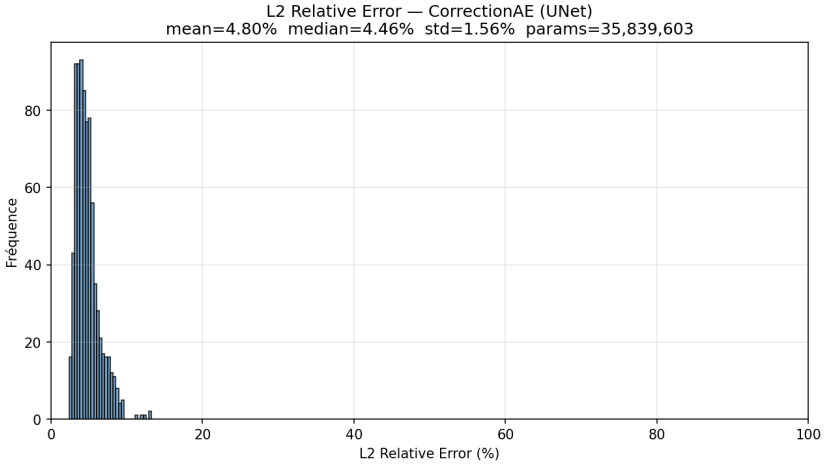


Figure 15: Relative L^2 error distribution after residual UNet correction (mean: 4.80%, median: 4.46%). The correction reduces error by approximately 76% compared to the baseline latent surrogate.

Method	Parameters	Mean (%)	Median (%)	Std (%)	Range (%)
LaplaceLatentModel (baseline)	35.36 M	19.86	18.50	5.35	[12.92, 48.80]
LaplaceLatentModel (finetuned)	35.36 M	10.19	9.30	2.93	[6.75, 23.18]
CorrectionAE (+ UNet)	35.84 M	4.80	4.46	1.56	[2.37, 13.28]

Table 2: Quantitative comparison of transient surrogates on the test set. The baseline latent surrogate achieves 19.86% mean error. End-to-end fine-tuning reduces this to 10.19% (48% improvement), while the UNet residual corrector further reduces error to 4.80% (76% improvement over baseline).

Key findings:

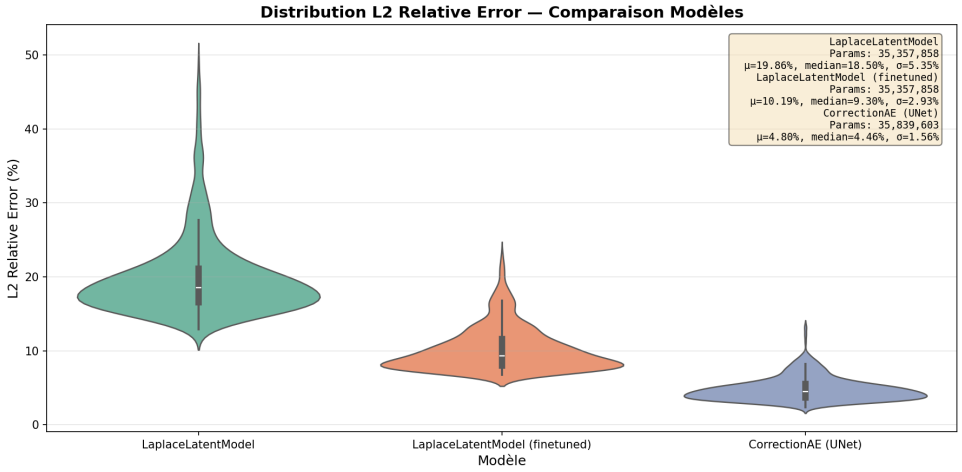


Figure 16: Violin plot comparison of prediction error distributions across all three models. The plot shows the median (horizontal line), quartiles (box), and full distribution (violin shape) for each method.

- **Fine-tuning impact.** End-to-end fine-tuning of the per-frequency MLPs and shared decoder reduces median error from 18.50% to 9.30%, a gain of approximately 50%. The reduced standard deviation (5.35% $\rightarrow$ 2.93%) indicates that fine-tuning not only improves mean performance but also makes predictions more consistent across the test set.
- **Residual correction.** The UNet residual corrector achieves median error of 4.46%, representing a further 52% reduction relative to the finetuned model. The error range tightens dramatically: from [6.75%, 23.18%] to [2.37%, 13.28%], with the tightest standard deviation (1.56%), indicating highly stable and predictable performance.
- **Parameter efficiency.** All three models possess comparable parameter counts ($\sim$ 35M), suggesting that the improvements stem from representation alignment and post-processing rather than architectural complexity. The UNet adds only 481 k trainable parameters for its substantial error reduction.
- **Robustness.** The monotone decrease in standard deviation across the pipeline (5.35% $\rightarrow$ 2.93% $\rightarrow$ 1.56%) demonstrates that each refinement step stabilises predictions, reducing outliers. The corrected model shows no test samples with error exceeding 13.28%, whereas the baseline exhibits errors above 48%.

4 Conclusion

This work presents a hierarchy of neural surrogate architectures for PDE emulation centred on learned latent representations. On the stationary problem, a VAE two-stage approach decouples representation learning from parameter regression. On the transient problem, Tucker POD and per-frequency SVD provide linear baselines, while a frequency-conditioned autoencoder with sinusoidal encoding and FiLM modulation learns a non-linear latent space shared across all Laplace frequencies. Per-frequency MLPs map the physical parameters into this latent space; end-to-end fine-tuning then aligns the surrogate with the decoder. A UNet residual corrector applied post hoc further reduces prediction error at marginal cost.

Future directions include incorporating physics-informed constraints into the latent space, extending the framework to varying source geometries, and applying the residual corrector to other reduced-order surrogate families.

References

[1] A. Ammar, M. Ben Saada, E. Cueto, and F. Chinesta. Casting hybrid twin: physics-based reduced order models enriched with data-driven models enabling the highest accuracy in real-time. *International Journal of Material Forming*, 17(2):16, 2024. doi:10.1007/s12289-024-01812-4.

[2] X. Fan, Y. Lin, B. Gong, and H. Li. Offline meteorology-pollution coupling global air pollution forecasting model with bilinear pooling. Preprint, n.d.

[3] Q. Guo, M. Ren, S. Wu, et al. Applications of artificial intelligence in the field of air pollution: a bibliometric analysis. *Frontiers in Public Health*, 10:933665, 2022. doi:10.3389/fpubh.2022.933665.

[4] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021. doi:10.1038/s42254-021-00314-5.

[5] C. Meng, S. Seo, D. Cao, S. Griesemer, and Y. Liu. When physics meets machine learning: a survey of physics-informed machine learning. Preprint, n.d.

[6] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[7] A. Quarteroni, P. Gervasio, and F. Regazzoni. Combining physics-based and data-driven models: advancing the frontiers of research with scientific machine learning. *Mathematical Models and Methods in Applied Sciences*, 35(4):905–1071, 2025. doi:10.1142/S0218202525500125.